VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA STROJNÍHO INŽENÝRSTVÍ

FACULTY OF MECHANICAL ENGINEERING

ÚSTAV MECHANIKY TĚLES, MECHATRONIKY A BIOMECHANIKY

INSTITUTE OF SOLID MECHANICS, MECHATRONICS AND BIOMECHANICS

MODULÁRNÍ SYSTÉM PRO AUTOMATIZOVANÉ ŘÍZENÍ MUZEJNÍCH EXPOZIC

MODULAR SYSTEM FOR THE AUTOMATED CONTROL OF MUSEUM EXHIBITS

BAKALÁŘSKÁ PRÁCE

BACHELOR'S THESIS

AUTOR PRÁCE Andrej Žabka

AUTHOR

VEDOUCÍ PRÁCE Ing. Michal Bastl, Ph.D.

SUPERVISOR

BRNO 2026

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ FAKULTA STROJNÍHO INŽENÝRSTVÍ

Zadání bakalářské práce

Ústav:	Ústav mechaniky těles, mechatroniky a biomechaniky
Student:	**Andrej Žabka**
Studijní program:	Mechatronika
Studijní obor:	bez specializace
Vedoucí práce:	**Ing. Michal Bastl, Ph.D.**
Akademický rok:	2025/26

Ředitel ústavu Vám v souladu se zákonem č.111/1998 o vysokých školách a se Studijním a zkušebním řádem VUT v Brně určuje následující téma bakalářské práce:

Modulární systém pro automatizované řízení muzejních expozic

Stručná charakteristika problematiky úkolu:

Student se bude zabývat návrhem a realizací modulárního systému pro automatizované řízení muzejních expozic. Úkolem je vytvořit řešení, které umožní spouštění časovaných scén v jedné místnosti a zajistí ovládání různých zařízení, jako je osvětlení, motory, aktuátory či speciální efekty. Systém bude založen na využití platformy Raspberry Pi jako centrálního řídicího prvku a modulů ESP32 jako distribuovaných periferních jednotek. Komunikace bude realizována prostřednictvím protokolu MQTT a strukturovaného formátu JSON, který umožní snadno definovat jednotlivé scény a jejich parametry. Výsledkem práce bude funkční vzorek systému pro jednu místnost, který bude demonstrovat možnosti navrženého řešení, jeho modularitu a připravenost k rozšíření.

Cíle bakalářské práce:

1. Provedení rešerše problematiky, zejména protokolu MQTT, formátu JSON a využití platforem Raspberry Pi a ESP32 v oblasti IoT.
2. Návrh architektury systému se zajištěním modularity a možnosti budoucího rozšíření.
3. Návrh a implementace hardwarové a softwarové části systému pro jednu místnost s možností spouštění scén.
4. Experimentální ověření funkčnosti systému na demonstrační úloze.

Seznam doporučené literatury:

[1] VALÁŠEK, Michael. Mechatronika. Praha: České vysoké učení technické, 1995. ISBN 80-01-01276-X.

[2] HEROUT, Pavel. Učebnice jazyka C. 1. vydání. Brno: Computer Press, 2004. ISBN 80-251-0452-3

[3] MALÝ, Martin. ESP32 prakticky : od základních obvodů k pokročilým aplikacím. 1. vydání. Praha : CZ.NIC, z.s.p.o., 2024. 525 s. ISBN 978-80-88168-79-9.

Termín odevzdání bakalářské práce je stanoven časovým plánem akademického roku 2025/26

V Brně, dne

L. S.

prof. Ing. Jindřich Petruška, CSc. ředitel ústavu	doc. Ing. Jiří Hlinka, Ph.D. děkan fakulty

Abstrakt

Tato bakalárska práca sa zaoberá návrhom a realizáciou modulárneho riadiaceho systému pre interaktívne múzejné expozície s využitím platforiem Raspberry Pi a ESP32...

Summary

This bachelor thesis deals with the design and implementation of a modular control system for interactive museum exhibits using Raspberry Pi and ESP32 platforms...

Klíčová slova

IoT, Raspberry Pi, ESP32, MQTT, múzeum, riadiaci systém, mechatronika

Keywords

IoT, Raspberry Pi, ESP32, MQTT, museum, control system, mechatronics

Bibliografická citace

Citace tištěné práce:
ŽABKA, A. *Modulární systém pro automatizované řízení muzejních expozic.* Bakalářská práce. Ing. Michal Bastl, Ph.D. (vedoucí práce). Brno: Vysoké učení technické v Brně, Fakulta strojního inženýrství, 2026.

Citace elektronického zdroje:
ŽABKA, A. *Modulární systém pro automatizované řízení muzejních expozic.* Online, bakalářská práce. Ing. Michal Bastl, Ph.D. (vedoucí práce). Brno: Vysoké učení technické v Brně, Fakulta strojního inženýrství, 2026. Dostupné z: https://www.vut.cz/studenti/zav-prace/detail/174036. [cit. 2026-03-07].

Týmto čestne vyhlasujem, že som bakalársku prácu vypracoval samostatne...

Andrej Žabka

Brno . . . .

Na tomto mieste by som chcel poďakovať svojmu vedúcemu práce za odborné vedenie...

Andrej Žabka

Obsah

1 Úvod **9**

2 Súčasný stav problematiky **10**
2.1 Architektúry riadiacich systémov 10
2.2 Existujúce prístupy k riadeniu expozícií 11
2.2.1 Priemyselné riadiace systémy (PLC/IPC) 11
2.2.2 Profesionálne show-control systémy 12
2.2.3 Otvorené IoT a SBC systémy (DIY / Maker / Embedded) 12
2.3 Komunikačné rozhrania a protokoly 13
2.3.1 Prenosové médiá: drôtové vs. bezdrôtové 13
2.3.2 Aplikačné protokoly pre IoT komunikáciu 13
2.4 Hardvérové platformy pre IoT 14
2.4.1 Jednodoskové počítače (SBC) 15
2.4.2 Mikrokontroléry pre koncové uzly 15
2.4.3 Výkonové prvky (aktuátory) 16
2.5 Dátové formáty pre konfiguráciu a prenos 16
2.6 Zhrnutie rešerše 17

3 Návrh architektúry systému **18**
3.1 Požiadavky na systém 18
3.2 Logická a sieťová koncepcia 18
3.3 Hardvérová voľba a sieťový prenos 19
3.4 Princíp softvérového jadra (Scene Engine) 20
3.5 Architektúra komunikačnej vrstvy 20

4 Implementácia hardvérovej časti **22**
4.1 Centrálna jednotka – Raspberry Pi 22
4.2 Distribuované uzly – ESP32 22
4.3 Výkonové prvky 23
4.3.1 Riadenie jednosmerných (DC) motorov 23
4.3.2 Spínanie sieťového napätia (230 V) a senzorika 24
4.4 Napájanie a zapojenie prototypu 24

5 Implementácia softvérovej časti **26**
5.1 Firmvér ESP32 26
5.2 Backend Raspberry Pi 27
5.3 Stavový automat a scene engine 27

5.4 Webový dashboard 27
5.5 Konfigurácia scén – JSON formát 28

6 Experimentálne overenie **29**
6.1 Popis demonštračnej úlohy (Setup) 29
6.2 Testované scenáre a výsledky 29
6.2.1 Scenár 1 – Komplexná lineárna scéna s hardvérom 29
6.2.2 Scenár 2 – Dlhodobý test stability riadiaceho systému 29
6.2.3 Scenár 3 – Test sieťovej odolnosti a LWT (Last Will) 30
6.3 Zhodnotenie overenia 30

7 Záver **31**

Zoznam obrázkov **32**

Zoznam tabuliek **33**

Použitá literatúra **34**

Použitá literatúra **34**

1 Úvod

Moderné [**plc_journal**] výstavníctvo prechádza v posledných rokoch výraznou zmenou. Návštevníci múzeí a galérií už nevyhľadávajú iba pasívne prezeranie exponátov, ale často očakávajú interaktívny prvok alebo príbehovú líniu, do ktorej môžu aktívne vstupovať. Takýto typ expozícií kladie zvýšené nároky na technickú infraštruktúru. Jednoduché prehrávanie videa v slučke a statické osvetlenie už nepostačujú. Riadiaci systém musí byť schopný reagovať na vstupy návštevníka, vetviť scénu podľa zvolených možností a synchronizovať jednotlivé prvky expozície v reálnom čase, vrátane zmeny osvetlenia, pohybu mechanických častí či spúšťania efektových zariadení.

Realizácia takýchto inštalácií však naráža na viacero technologických a finančných obmedzení. Komerčné show-control riešenia ponúkajú vysokú spoľahlivosť a funkcionalitu, avšak ich obstarávacie náklady sú pre menšie expozície často neúnosné. Jednoduché časovače a lokálne ovládače naopak neposkytujú potrebnú úroveň interaktivity a flexibility. Problematická je aj kabeláž – v historických objektoch býva ťažké, prípadne nežiadúce viesť desiatky metrov vodičov k senzorom a aktuátorom.

Cieľom tejto bakalárskej práce je návrh a implementácia riadiaceho systému, ktorý uvedené problémy rieši kombináciou IoT technológií a vlastnej softvérovej architektúry. Pôvodný koncept bol založený na jednoduchom lineárnom spúšťači efektov. Analýza požiadaviek však ukázala potrebu komplexnejšieho riešenia, ktoré umožňuje vetvenie scén a reakciu na externé udalosti. Výsledkom je systém postavený na stavovom automate, ktorý umožňuje definovať priebeh expozície presnejšie a flexibilnejšie.

Navrhovaná architektúra využíva bezdrôtový model klient–server, ktorý minimalizuje potrebu rozsiahlej kabeláže. Centrálnu riadiacu logiku zabezpečuje minipočítač Raspberry Pi. Ten komunikuje s distribuovanými uzlami na platforme ESP32 prostredníctvom protokolu MQTT. Správanie systému nie je pevne zakompilované vo firmvéri, ale je konfigurovateľné pomocou JSON súborov. Táto vlastnosť umožňuje jednoducho meniť scenár expozície bez zásahu do zdrojového kódu. Funkčnosť systému je v závere práce demonštrovaná na prototypovej miestnosti, ktorá integruje riadenie motorov, osvetlenia, dymostroja a multimediálneho obsahu.

2 Súčasný stav problematiky

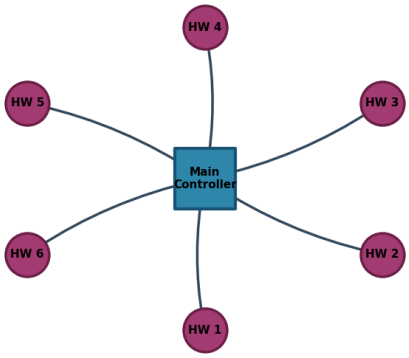
Cieľom tejto kapitoly je analyzovať súčasný stav technických prostriedkov používaných v modernom výstavníctve, multimediálnych inštaláciách a automatizácii. Kapitola poskytuje kritický prehľad dostupných hardvérových a softvérových platforiem s dôrazom na ich vhodnosť pre realizáciu interaktívnych expozícií. Zároveň definuje kľúčové technické požiadavky na komunikačné protokoly v distribuovaných systémoch.

2.1 Architektúry riadiacich systémov

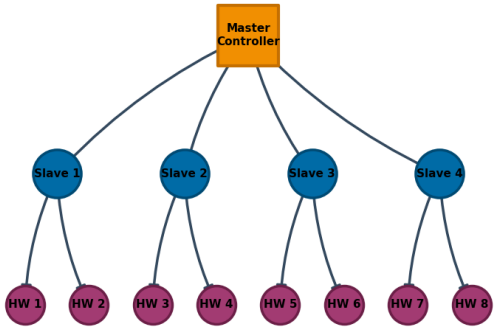
Pri návrhu riadiacich systémov pre expozície je možné zvoliť rôzne topológie, ktoré definujú vzťah medzi riadiacou logikou a výkonnými prvkami. V praxi sa stretávame predovšetkým s dvoma základnými prístupmi:

- **Centralizovaná topológia:** Je to taká architektúra, kde jedna centrálna jednotka priamo ovláda všetky vstupné a výstupné periférie. Topológia zvyčajne využíva zapojenie typu hviezda, čo znamená vedenie individuálnej kabeláže od každého prvku až do centrálneho rozvádzača. Výhodou je jednoduchá správa na jednom mieste, avšak nevýhodou je závislosť na jedinom bode zlyhania (centrále) a komplikácie s rozsiahlym vedením kabeláže z jedného uzla.

- **Distribuovaná topológia (Master-Slave):** V tomto modeli je inteligencia rozložená do viacerých uzlov. Centrálna jednotka (master) komunikuje s ďalšími periférriami (slaves), ktoré riešia svoju vlastnú časť logiky.Toto umožňuje rozdelenie zodpovednosti za riadenie jednotlivých častí expozície medzi viacero zariadení. Tento prístup je typický pre IoT systémy, kde komunikácia prebieha prostredníctvom sieťových protokolov a periférie môžu obsahovať vlastné mikrokontroléry. Topológia tak umožňuje modulárne rozširovanie systému ale aj redukuje potrebu ťahať kabeláž z jedneho riadiaceho bodu na všetky periférie, ktoré môžu byť umiestnené v rôznych častiach expozície. Distribuované riešenie môže byť aj odolnejšie voči zlyhaniu – výpadok jednej periférie neparalyzuje celý systém.



Obrázok 2.1: Centralizovaná topológia



Obrázok 2.2: Distribuovaná topológia

2.2 Existujúce prístupy k riadeniu expozícií

V súčasnosti neexistuje jedno univerzálne riešenie pre implementáciu interaktívnych expozícií. Realizátori si spravidla vyberajú medzi viacerými kategóriami, z ktorých každá má svoje špecifiká, výhody a obmedzenia. Výber konkrétnej platformy závisí od požiadaviek na spoľahlivosť, synchronizáciu multimédií a dostupný rozpočet.

2.2.1 Priemyselné riadiace systémy (PLC/IPC)

V oblasti klasickej automatizácie a priemyslu sú dlhoročným štandardom Programovateľné logické automaty (PLC) od výrobcov ako **Siemens, Beckhoff či Mitsubishi**, prípadne priemyselné počítače (IPC) v kombinácii so SCADA systémami. Tieto systémy sú navrhnuté primárne pre výrobné linky a procesné riadenie, kde je prioritou deterministické správanie a nepretržitá prevádzka.

Charakteristika a výhody:

- **Vysoká spoľahlivosť a robustnosť:** PLC sú konštruované na prevádzku v náročných podmienkach (prach, vibrácie, teplotné výkyvy) a vykazujú extrémne nízku poruchovosť.
- **Determinizmus:** Programovanie prebieha podľa normy IEC 61131-3 (Ladder Diagram, Structured Text), čo zaručuje, že reakcia systému na vstup nastane v garantovanom čase.
- **Centralizovaná topológia:** Využíva sa zapojenie typu hviezda, čo uľahčuje správu na jednom mieste (rozvádzač), no zvyšuje nároky na kabeláž.

Obmedzenia pre multimediálne expozície: Nasadenie PLC v moderných múzeách naráža na limity v oblasti spracovania audiovizuálneho obsahu:

- **Absencia podpory multimédií:** PLC sú určené na spracovanie logických signálov (I/O) a nedisponujú rozhraniami pre grafický výstup (HDMI) ani zvuk. Prehrávanie obsahu vyžaduje externé prehrávače, ktoré PLC iba spúšťa (triggeruje).
- **Cena a inštalácia:** Vysoká cena priemyselných komponentov a nutnosť viesť kabeláž od každého zariadenia k centrále robí toto riešenie neefektívnym pre menšie alebo dočasné inštalácie.

2.2.2 Profesionálne show-control systémy

Táto kategória predstavuje súčasný štandard v profesionálnom výstavníctve, tematických parkoch a koncertných halách. Ide o špecializované systémy navrhnuté priamo na riadenie zážitku. Medzi popredných výrobcov patria **Medialon, Alcorn McBride, Crestron či AMX**.

Na rozdiel od priemyselných PLC, ktoré pracujú v cykloch, show-control systémy pracujú na princípe časovej osi (**timeline**). Umožňujú operátorovi definovať sekvenciu udalostí v čase, podobne ako v strihovom softvéri.

Technické špecifiká a protokoly: Tieto systémy natívne integrujú protokoly používané v zábavnom priemysle:

- **SMPTE Timecode:** Pre presnú časovú synchronizáciu medzi zvukom, obrazom a fyzickými efektmi.
- **DMX512 / Art-Net:** Pre komplexné riadenie scénického osvetlenia.
- **OSC (Open Sound Control):** Pre sieťovú komunikáciu s multimediálnymi servermi (napr. Resolume, Watchout).
- **Synchronizácia:** Systém dokáže presne zosúladiť spustenie videa na viacerých projektoroch s pohybom animatroniky a zmenou svetelnej atmosféry.

Nevýhody: Hlavnou bariérou nasadenia sú vysoké náklady a uzavretosť ekosystému. Ide o proprietárne riešenia, ktoré vytvárajú závislosť na dodávateľovi. Akékoľvek rozšírenie systému alebo zmena logiky často vyžaduje licencovaný softvér a certifikovaného technika, čo je pre nízkonákladové projekty neudržateľné.

2.2.3 Otvorené IoT a SBC systémy (DIY / Maker / Embedded)

Treťou, čoraz populárnejšou kategóriou v modernom výstavníctve, sú otvorené a modulárne systémy postavené na princípoch Internetu vecí (IoT). Na rozdiel od priemyselných PLC alebo uzavretých Show-Control systémov, tento prístup nevyužíva proprietárny hardvér jedného výrobcu, ale kombinuje bežne dostupné výpočtové platformy s open-source softvérom.

Tento smer predstavuje konvergenciu klasickej automatizácie a informačných technológií (IT). Systém nie je definovaný fyzickým zariadením, ale softvérovou architektúrou, ktorá umožňuje flexibilné prepojenie rôznorodých komponentov.

Hardvérová koncepcia: Architektúra je typicky rozdelená na dve funkčné úrovne:

- **Výkonné centrálne jednotky (SBC):** Využívajú sa jeddoskové počítače s plnohodnotným operačným systémom (najčastejšie na báze Linuxu). Tieto slúžia na koordináciu celého systému, správu databáz a predovšetkým na náročné multimediálne úlohy, ako je prehrávanie videa vo vysokom rozlíšení či generovanie grafiky, čo je oblasť, kde klasické PLC zaostávajú.
- **Distribuované periférie:** Koncové prvky (senzory, osvetlenie, mechanika) sú riadené samostatnými mikrokontrolérmi. Tie neposielajú do centra len surové elektrické signály, ale fungujú ako inteligentné sieťové uzly, ktoré spracúvajú dáta lokálne a komunikujú bezdrôtovo alebo po lokálnej sieti.

Softvérová flexibilita: Kľúčovou vlastnosťou tohto prístupu je nezávislosť od uzatvorených vývojových prostredí. Riadiaca logika je implementovaná v štandardných programovacích jazykoch, čo vývojárom poskytuje neobmedzené možnosti pri tvorbe interakcií. Namiesto špeciálnych priemyselných zberníc sa pre komunikáciu využívajú štandardné sieťové technológie a protokoly známe z webového vývoja, čo zjednodušuje integráciu s tabletmi, webovými stránkami či vzdialenou správou.

V tejto kategórii sa často uplatňujú aj vizuálne nástroje na orchestráciu toku dát (napr. Node-RED), ktoré slúžia na rýchle prepojenie služieb, zatiaľ čo kritická logika zostáva v kóde.

Zhodnotenie prístupu: Otvorené systémy ponúkajú bezkonkurenčný pomer ceny a výkonu a maximálnu prispôsobiteľnosť. Umožňujú realizovať komplexné scenáre, ktoré by boli v komerčných systémoch neúmerne drahé. Daňou za túto slobodu je však presun zodpovednosti na realizátora – vyžaduje sa hlbšia znalosť softvérovej architektúry a sieťovej komunikácie, keďže integrátor si systém stavia a garantuje sám.

2.3 Komunikačné rozhrania a protokoly

Pre zabezpečenie výmeny dát v distribuovanom systéme je kľúčová voľba prenosového média a komunikačného protokolu. Interaktívne inštalácie často zahŕňajú množstvo senzorov a akčných členov, ktoré musia navzájom spoľahlivo komunikovať v reálnom čase, preto sa detailne zvažujú možnosti prepojenia komponentov.

2.3.1 Prenosové médiá: drôtové vs. bezdrôtové

Pri inštaláciách v múzeách je kritickým faktorom spôsob prepojenia komponentov. Tradičné drôtové zbernice (RS-485, Ethernet) ponúkajú vysokú spoľahlivosť, odolnosť voči rušeniu a deterministickú komunikáciu potrebnú pre časovo citlivé aplikácie. Ich nevýhodou je nutnosť fyzickej kabeláže ku každému prvku, čo môže byť v historických priestoroch problematické a limituje modularitu – presun či pridanie exponátu si vyžaduje nové káblové rozvody.

Bezdrôtové technológie (Wi-Fi, Bluetooth, Zigbee) eliminujú potrebu kabeláže a výrazne zvyšujú flexibilitu systému. Mikrokontroléry ESP32 majú vstavané Wi-Fi a priamo sa pripájajú do lokálnej siete, čo umožňuje jednoducho premiestňovať interaktívne prvky bez nových prípojok. Zriadenie Wi-Fi pokrytia je neinvazívne a ideálne pre modulárne či dočasné expozície.

Nevýhodou je nižšia spoľahlivosť – bezdrôtová komunikácia je náchylnejšia na rušenie (hrubé múry, kovové konštrukcie, zahlteté Wi-Fi spektrum). Pre kritické časované sekvencie je preto vhodné využiť dedikovanú internú Wi-Fi sieť oddelenú od verejného prístupu.

Celkovo však bezdrôtové riešenia v prostredí múzea prinášajú voľnosť. IoT čipy (ako spomínaný ESP32) majú integrovanú podporu Wi-Fi, takže priamo komunikujú cez internetovú sieť. To umožňuje budovať plne distribuovanú architektúru bez centrálnej kabeláže. Drôtové spojenie zostáva vhodné pre špecifické prípady, kde je potrebná garantovaná odozva bez rizika rušenia.

2.3.2 Aplikačné protokoly pre IoT komunikáciu

Voľba komunikačného protokolu musí zohľadňovať obmedzený výkon mikrokontrolérov, požiadavky na nízku latenciu a spoľahlivosť prenosu. V oblasti IoT sa najčastejšie porovnávajú tieto varianty:

HTTP (request/response) funguje na modeli klient-server, kde zariadenie musí aktívne dopytovať server o zmenu stavu (polling). Hoci je HTTP všadeprítomný a jednoducho použiteľný, pre aplikácie v reálnom čase je neefektívny – neustále dotazovanie vytvára zbytočnú dátovú záťaž a prináša latenciu.

WebSocket umožňuje udržiavať otvorený obojsmerný komunikačný kanál, čo odstraňuje potrebu pollingu – server vie aktívne poslať správu klientovi (push). Používa sa v real-time webových aplikáciách, no je relatívne ťažkopádny pre najjednoduchšie mikrokontroléry a neštandardný v prostredí MCU.

MQTT (publish/subscribe) je odľahčený protokol navrhnutý priamo pre IoT, pracujúci nad TCP/IP. Funguje na architektúre sprostredkovateľa (broker) a klientov:

- Každý klient sa môže prihlásiť na odber určitej témy (topic) alebo publikovať správu na tému
- Broker zabezpečí distribúciu správ všetkým prihláseným klientom
- Architektúra úplne oddeľuje vysielateľa od prijímateľa
- Umožňuje model komunikácie (viacero zdrojov → viacero prijímateľov)

MQTT bol navrhnutý na minimalizáciu dátovej režie – každá správa nesie len malú hlavičku (pevne 2 bajty), zvyšok je užitočný obsah. Na rozdiel od neustáleho pollingu pri HTTP sa správy posielajú len vtedy, keď nastane zmena alebo udalosť. MQTT udržiava trvalé pripojenie medzi klientom a brokerom, pričom funkcia *Last Will and Testament* umožňuje detekovať výpadok klienta a informovať o ňom ostatných.

Protokol definuje tri úrovne kvality služby (QoS), ktoré umožňujú prispôsobiť komunikáciu podľa požiadaviek:

Tabuľka 2.1: Úrovne kvality služby (QoS) v MQTT

QoS	Úroveň	Použitie
0	At most once	Menej kritické dáta (napr. pravidelné senzorické hodnoty), bez potvrdenia
1	At least once	Dôležité správy, možné duplikáty, s potvrdením
2	Exactly once	Kritické povely (napr. núdzové zastavenie), garantované presne raz doručenie

MQTT si v súčasnosti získal mimoriadnu popularitu v IoT komunite a stal sa v podstate štandardom pre malé zariadenia, čo z neho robí vhodného kandidáta aj pre účel riadenia expozície.

2.4 Hardvérové platformy pre IoT

Realizácia riadiaceho systému vyžaduje výber hardvéru pre dve úrovne: centrálnu jednotku (koordináciu systému a multimediálne funkcie) a koncové uzly (senzory, aktuátory, periférie v priestore expozície).

2.4.1 Jednodoskové počítače (SBC)

Pre centrálnu jednotku, ktorá zabezpečuje koordináciu celého systému, spracovanie komplexnej logiky a multimediálne funkcie (prehrávanie videí, zvukov), sa javia najvhodnejšie jednodoskové počítače. Ide o plnohodnotné miniatúrne počítače s procesorom, pamäťou a všetkými bežnými rozhraniami, schopné prevádzkovať operačný systém (typicky Linux).

Referenčnou platformou v tejto kategórii je **Raspberry Pi** – populárny nízkonákladový SBC s veľkou komunitou. Raspberry Pi ponúka dostatočný výkon na beh OS Linux (resp. Raspberry Pi OS) pri kompaktných rozmeroch a nízkej spotrebe (jednotky wattov). Pre múzejné účely je dôležité, že má integrované grafické výstupy (HDMI), audio výstup, USB porty a v novších modeloch aj Wi-Fi a Bluetooth modul.

V kontexte múzejných inštalácií už Raspberry Pi preukázal svoju vhodnosť. Dôvodom je najmä pomer ceny a výkonu: v porovnaní s priemyselnými PC alebo PLC je cena rádovo nižšia, pričom výkon (štvorjadrové CPU, GPU schopné prehrávať 1080p/4K video) je pre účely interaktívnych displejov plne dostačujúci. Zariadenie nemá pohyblivé časti (pasívne chladené), čo zaručuje spoľahlivú nepretržitú prevádzku. Navyše, Raspberry Pi disponuje 40-pinovým GPIO headerom, cez ktorý možno pripojiť rôzne senzory a aktuátory priamo.

Ďalším plusom je obrovský ekosystém otvoreného softvéru – od ovládačov pre hardware, cez knižnice na prácu s videom/zvukom, až po celé platformy pre IoT a automatizáciu (Node-RED, Home Assistant). Z vývojárskeho hľadiska je výhodou, že na Raspberry Pi beží plnohodnotný Linux, takže je možné vyvíjať v skriptovacích jazykoch (Python, Node.js) a využiť štandardné nástroje, čo značne urýchľuje prototypovanie.

2.4.2 Mikrokontroléry pre koncové uzly

Koncové uzly v expozícii predstavujú jednotlivé interaktívne prvky – senzory (pohybové čidlá, dotykové tlačidlá) a aktuátory (motory, relé spínače svetiel). Tieto uzly musia samostatne vykonávať jednoduchú logiku a komunikovať s centrálnym systémom.

Arduino a 8-bitové mikrokontroléry: Klasické Arduino Uno (ATmega328) reprezentuje staršiu generáciu 8-bitových mikrokontrolérov s taktom 16 MHz, malou pamäťou (2 kB RAM, 32 kB flash) a obmedzenými možnosťami pripojenia. Arduino vyniklo jednoduchosťou a množstvom rozširujúcich modulov. Pre účely IoT však nedisponuje natívnou sieťovou konektivitou – potrebujeme Ethernet shield alebo Wi-Fi modul (napr. ESP8266), čo zvyšuje cenu a zložitosť. Spracovať TCP/IP stack na 16 MHz procesore s 2 kB RAM je na hranici možností.

Moderné 32-bitové mikrokontroléry (ESP32): V posledných rokoch sa v komunite IoT mimoriadne rozšíril čip ESP32 od firmy Espressif. Ide o výkonný 32-bitový systém-na-čipe, ktorý v sebe integruje Wi-Fi (802.11 b/g/n) a Bluetooth modul, dvojjadrový procesor s taktom až 240 MHz a pamäť 520 kB SRAM. ESP32 kombinuje v jednom lacnom module všetko potrebné pre samostatný IoT uzol – výpočtový výkon aj bezdrôtovú komunikáciu.

Oproti Arduinu ponúka ESP32 vyšší výkon o dva rády (240 MHz vs. 16 MHz), viacvláknové spracovanie (2 jadrá) a bohatšie periférie (viac ADC kanálov, DAC prevodníky,

kapacitné dotykové vstupy). Pre náš účel je dôležité, že má integrované Wi-Fi, takže pripojenie do MQTT siete je priamočiare bez ďalšieho hardvéru. Pamäť 520 kB RAM dovolí spracovávať aj textové protokoly (JSON správy, fronty MQTT správ).

Programovať ESP32 možno v prostredí Arduino (C++), cez platformu ESP-IDF (C, C++) alebo MicroPythonom, čo dáva flexibilitu vývoja. Existuje množstvo vývojových dosiek (DevKitC, NodeMCU, LOLIN32) uľahčujúcich prácu s týmto čipom.

2.4.3 Výkonové prvky (aktuátory)

Okrem inteligentných jednotiek bude systém obsahovať aj výkonové komponenty, ktoré priamo ovládajú fyzické zariadenia: elektromotory, osvetlenie na 230 V, prípadne iné efektové prvky.

H-mostíky pre DC motory: Na riadenie smeru otáčania a rýchlosti jednosmerných motorov sa používajú H-bridge driver moduly. Staršie integrované obvody ako L293D alebo L298N majú značné nedostatky – v dôsledku použitia Darlington tranzistorov strácajú niekoľko voltov a pri väčších prúdoch sa výrazne zahrievajú. Moderné H-mostíky prešli na MOSFET tranzistory s nízkym odporom v sepnutom stave, čím sa minimalizuje úbytok napätia a teplo. K dispozícii sú kompaktné driver čipy (napr. TB6612FNG, VNH2SP30, drv88xx séria) schopné dodávať aj desiatky ampérov pri zanedbateľnom zahrievaní.

Spínanie sieťového napätia (230 V): V expozíciách často potrebujeme ovládať svetelné okruhy, dymostroj či iné zariadenia na sieťové napájanie 230 VAC. Tradičným riešením je použitie elektromechanických relé, ktoré však pri zopnutí vydáva klikavý zvuk a má obmedzenú životnosť. Preto sa v moderných inštaláciách preferujú polovodičové relé (SSR, Solid State Relay). SSR využíva elektronické spínacie prvky (triaky, tyristory) a nemá pohyblivé časti. Výhody sú: tichá prevádzka (žiadne klikanie), rýchle spínanie aj vysokých frekvencií a dlhá životnosť (nepodlieha mechanickému opotrebovaniu). Pre striedavé zaťaženie 230 V sa používajú najmä SSR s triakom, ktoré spínajú záťaž pri prechode napätia nulou (zero-cross switching) – to redukuje rušenie v sieti.

2.5 Dátové formáty pre konfiguráciu a prenos

Pre štruktúrovanie prenášaných dát a ukladanie konfigurácie scén je nutné zvoliť vhodný formát. V kontexte IoT a webových technológií sa najčastejšie porovnávajú tieto textové formáty:

XML (eXtensible Markup Language) je dlhoročným priemyselným štandardom pre výmenu dát. Výhodou je možnosť definovať vlastnú schému (XSD) pre validáciu a hierarchickú štruktúru s tagmi a atribútmi. Pre IoT zariadenia je však nevýhodný – každá položka je ohraničená značkami `<tag>...</tag>`, čo výrazne zvyšuje objem prenášaných dát.

YAML (YAML Ain't Markup Language) je moderný formát obľúbený v developerských komunitách (Ansible, Docker Compose, Kubernetes). Je dizajnovaný tak, aby bol čitateľný pre človeka – používa odsadenie medzerami namiesto zátvoriek na vyjadrenie hierarchie a podporuje komentáre. Pre použitie v IoT má však dve hlavné nevýhody: je citlivý na formátovanie (extra medzera môže zhodiť celý súbor) a knižnice pre mikrokontroléry nie sú tak rozšírené ani optimalizované ako pre iné formáty. YAML parser je komplexný kvôli podpore rôznych dátových typov a spravidla pomalší na spracovanie ako JSON. [CITACE]

JSON (JavaScript Object Notation) je aktuálne jeden z najpoužívanejších formátov na webe aj v IoT. Vznikol ako ľahšia alternatíva k XML a využíva syntaxu párov „kľúč: hodnota“ so zloženými zátvorkami `{}` pre objekty a hranatými zátvorkami `[]` pre polia. JSON si získal obľubu pre svoju jednoduchosť a stručnosť – na rozdiel od XML neobsahuje ukončovacie tagy, čo znamená menší objem prenášaných dát.

Z pohľadu mikrokontrolérov je JSON najpriaznivejší: ignoruje formatovanie (whitespace), takže odriadkovanie či odsadenie neovplyvní správnosť. Podporuje len základné dátové typy (reťazec, číslo, pole, objekt, boolean, null), čo zjednodušuje implementáciu parsera. Pre JSON existujú vysoko optimalizované knižnice prakticky vo všetkých jazykoch – v C/C++ (ArduinoJson), v Python (modul `json`) a mnohé ďalšie.

2.6 Zhrnutie rešerše

Analýza dostupných technických riešení preukázala, že pre špecifické potreby modulárnej, flexibilnej a cenovo dostupnej muzeálnej inštalácie nie sú optimálne ani robustné priemyselné PLC (kvôli vysokej cene a nutnosti centralizovanej kabeláže), ani uzavreté komerčné systémy „Show Control“.

3 Návrh architektúry systému

Táto kapitola popisuje celkovú architektúru navrhovaného riadiaceho systému. Vychádza z požiadaviek definovaných v predchádzajúcej kapitole a z analýzy dostupných technológií. Popisuje hardvérovú topológiu, softvérovú štruktúru, komunikačnú vrstvu a formát konfiguračných súborov.

3.1 Požiadavky na systém

Táto kapitola definuje hranice projektu a podmienky, ktoré musí výsledný riadiaci systém spĺňať pre zabezpečenie spoľahlivej prevádzky v prostredí expozície.

Z hľadiska **funkčných požiadaviek** sa očakáva najmä bezproblémová integrácia a synchronizácia rôznorodých hardvérových prvkov:

- **Riadenie motora a výkonu:** Primárnou úlohou je riadenie výkonových prvkov, špecificky jednosmerných motorov s riadením ich rýchlosti a smeru otáčania. Ďalšou kľúčovou funkciou je priame spínanie záťaží pracujúcich na sieťovom napätí **230 V** (osvetlenie, dymostroje) a taktiež **12 V**.
- **Multimediálna synchronizácia:** Keďže moderné expozície silne stavajú na audiovizuálnych vnemoch, systém musí zabezpečiť synchronizované prehrávanie zvuku a videa priamo z vlastného úložiska.
- **Interaktivita a časová os (*timeline*):** Systém musí vedieť na základe hardvérových vstupov zmeniť vetvenie prebiehajúcej scény, no zároveň podporovať chronologické časovanie sekvencií.

Medzi **nefunkčné požiadavky** s ohľadom na udržateľnosť patria:

- **Modularita a softvérové oddelenie:** Pre zachovanie oddelenia softvérového inžinierstva od obsahovej tvorby sa úprava logiky (scenára) musí dať vykonať výhradne modifikáciou **textových konfiguračných súborov**. Tým sa odstraňuje nutnosť zásahu do zdrojových kódov Pythonu.
- **Dostupnosť (Cost-effectiveness):** Vzhľadom na finančné limity inštalácií musí byť systém postavený na komerčne dostupných, cenovo výhodných mikrokontroléroch a **Internet of Things (IoT)** platformách namiesto drahých priemyselných automatov (PLC).

3.2 Logická a sieťová koncepcia

Na architektúru navrhovaného systému je nevyhnutné nazerať z dvoch odlišných, no vzájomne sa dopĺňajúcich hľadísk: z **logického (výpočtového)** a z **fyzického (sieťového)**.

Z logického hľadiska ide o **distribuovanú architektúru**. Výpočtový výkon a inteligencia nie sú sústredené výhradne v jednom centrálnom procesore, ale sú rozložené do

priestoru priamo k výkonovým prvkom. Koncové periférie postavené na mikrokontroléroch neslúžia len ako pasívne spínače. Naopak, aktívne dekódujú prichádzajúce textové správy (parsovanie reťazcov MQTT payloadu) a priamo generujú špecifické signály (napr. pulznošírkovú moduláciu pre plynulé riadenie otáčok motorov či jednoduché zapnutie/vypnutie spotrebičov pomocou relé.).

Z fyzického a sieťového hľadiska však systém prísne dodržiava **hviezdicovú topológiu (*Star topology*)**. Hoci sú distribuované uzly plne inteligentné, nekomunikujú vzájomne medzi sebou (netvoria *Mesh* sieť). Akýkoľvek dátový tok na sieti vedie striktne obojsmerne len cez centrálny uzol, ktorým je Wi-Fi prístupový bod so sprevádzkovaným MQTT sprostredkovateľom.

[MIESTO PRE OBRÁZOK: Bloková schéma fyzickej topológie hviezdicovej siete s odlíšením centrálneho uzla a okrajových distribuovaných periférií]

3.3 Hardvérová voľba a sieťový prenos

Systém je rozdelený do dvoch samostatných kategórií: na jeden centrálny uzol a na sieť výkonných periférií. Z hľadiska spoľahlivosti a rýchlosti prenosu medzi nimi by bolo ideálne zabezpečiť komunikáciu výhradne plne drôtovým spojením. V praxi však v expozíciách či *escape room* inštaláciách často nie je technologicky alebo esteticky možné naťahovať dodatočné káblové zväzky naprieč miestnosťou. Z tohto dôvodu bolo na plošnú distribúciu dátových príkazov z centrály zvolené bezdrôtové Wi-Fi pripojenie.

Dvojúrovňová architektúra bola fyzicky obsadená nasledovným hardvérom:

- **Centrálny uzol („Mozog“):** Hlavnú riadiacu jednotku systému tvorí jednodoskový počítač **Raspberry Pi**. Tento počítač plní úlohu centrálneho uzla, v ktorom je implementovaná hlavná logika riadenia celej miestnosti. Zvolený bol najmä pre svoj dostatočný výpočtový výkon a pamäť, ktoré umožňujú spoľahlivú prevádzku riadiaceho softvéru aj prehrávanie multimediálneho obsahu. Zariadenie zároveň poskytuje natívne rozhrania pre výstup obrazu (HDMI) a zvuku, čo umožňuje priamu obsluhu audio-vizuálnych prvkov expozície. Výhodou je aj dostupnosť sieťových rozhraní (*Wi-Fi* aj *Ethernet*), vďaka ktorým môže systém komunikovať s ostatnými zariadeniami a súčasne hostovať webový ovládací *dashboard* určený pre obsluhu.

- **Distribuované uzly („Svaly a zmysly“):** Na plnenie lokálnych fyzických úkonov slúži modulárna sieť bezdrôtových mikrokontrolérov **ESP32**, ktoré sú implementované a napájané priamo pri interaktívnych objektoch. Tieto uzly sa na základe svojho zamerania softvérovo rozdeľujú na:
  - **Senzorický uzol** pre hardvérovú detekciu vstupov od používateľov.
  - **Motorický uzol** pre obsluhu fyzického pohybu exponátov (jednosmerné motory).
  - **Reléový uzol** pre spínanie efektov a osvetlenia či iných zariadení pracujúcich na 12 V a sieťovom napätí 230 V.

3.4 Princíp softvérového jadra (Scene Engine)

Ústrednou požiadavkou na softvérové jadro bola schopnosť **vetviť priebeh expozície** na základe reálnych podnetov od návštevníka. Scenár preto nesmel byť pevnou lineárnou sekvenciou akcií — systém musel v ľubovoľnom momente vedieť čakať na vonkajší vstup (napr. stlačenie tlačidla) a podľa jeho výsledku dynamicky zvoliť ďalší smer deja. Klasický prístup blokujúceho vykonávania kódu je na takúto úlohu nevhodný, keďže vlákno čakajúce na jednu udalosť nie je schopné súčasne reagovať na iné podnety z prostredia.

Z tohto dôvodu bolo softvérové jadro navrhnuté ako neblokujúci **Konečný stavový automat (FSM — *Finite State Machine*)**. Priebeh expozície je rozdelený do samostatných **stavov**, pričom systém nepretržite sleduje podmienky prechodu — vypršanie časovača, dokončenie prehrávania média, alebo príchod správy od hardvérovej periférie — a podľa nich plynulo presúva chod scenára do ďalšieho stavu. Podrobná softvérová implementácia tejto architektúry je rozpracovaná v Kapitole 5.

[MIESTO PRE OBRÁZOK: Diagram konečného stavového automatu (FSM) – prechody z onEnter do timeline a čakanie na transition eventy]

3.5 Architektúra komunikačnej vrstvy

Pre výmenu dát medzi centrálnym automatom (v jazyku Python) a distribuovanými periférami (ESP32) bol zvolený **protokol MQTT** (*Message Queuing Telemetry Transport*). Prenos správ prebieha na koncepte *Publish/Subscribe* cez trvalé TCP/IP spojenie s centrálnym sieťovým sprostredkovateľom (*brokerom*) — čo zaručuje nízku réžiu a spoľahlivé doručenie v rámci lokálnej siete.

Navrhnutý model siete implementuje niekoľko kľúčových komunikačných štandardov:

- **Hierarchické adresovanie zariadení:** Každé zariadenie je adresované vlastným topikom v hierarchickom formáte (napr. `room1/motor1`). Raspberry Pi zverejní príkaz na daný topik; ESP32 mikrokontrolér, ktorý má tento topik odoberaný, správu prijme a vykoná požadovanú akciu.

- **Spätná väzba (*feedback*):** Po vykonaní príkazu periférny uzol odošle potvrdzovaciu správu späť na topik s príponou `/feedback` (napr. `room1/motor1/feedback`). Centrála tak vie, že príkaz bol skutočne doručený a fyzicky vykonaný.

- **Detekcia dostupnosti (status + LWT):** Každý ESP32 uzol pri pripojení aktívne zverejní stav `online` na topiku `devices/<id>/status` ako zostatkovú (*retained*) správu — centrála tak vždy pozná aktuálny stav siete. Súčasne je nastavená správa *Last Will and Testament* (LWT), ktorú broker automaticky zverejní ako `offline` na rovnakom topiku v prípade neočakávaného výpadku spojenia.

- **Konfigurácia scény ako externý JSON súbor:** Celá logika scenára — čo má každý exponát vykonať, v akom poradí a za akých podmienok — je definovaná

výhradne v externom súbore `.json`, nie priamo v kóde enginu. Obsluha môže scenár upravovať alebo vymeniť za behu programu bez akéhokoľvek zásahu do zdrojového kódu.

Vďaka tomuto oddeleniu scenára od enginu sa konfigurácia výstavy stáva samostatným textovým dokumentom zrozumiteľným aj pre netechnický personál.

[MIESTO PRE OBRÁZOK: Schéma toku MQTT správ (Publish/Subscribe) medzi Raspberry Pi a koncovými ESP32 uzlami so spätnou väzbou]

4 Implementácia hardvérovej časti

Táto kapitola je venovaná praktickej elektronickej a fyzickej inštalácii celého riadiaceho systému so zreteľom na inžiniersku spoľahlivosť a bezpečnosť.

4.1 Centrálna jednotka – Raspberry Pi

Na prepojenie softvéru a lokálnej reálnej vrstvy bol zvolený nasledujúci hardvér:

- **Umiestnenie a konektivita:** Centrálny minipočítač Raspberry Pi je fyzicky umiestnený v tesnej blízkosti audiovizuálnej techniky (napríklad priamo za monitorom/televízorom). Toto riešenie minimalizuje dĺžku káblov, keďže priamo zabezpečuje natívne prehrávanie videa cez HDMI a distribúciu zvuku do reproduktorov, pričom stačí prístup do rovnakej lokálnej siete.

- **Konfigurácia operačného systému:** Nasadenie „headless“ distribúcie Raspberry Pi OS Lite (bez grafického prostredia) pre ušetrenie systémových prostriedkov. Výpočtový výkon je tak striktne dedikovaný pre chod MQTT brokera `Mosquitto` a riadiaceho softvérového jadra.

- **Správa prístupových práv:** Systémové aplikácie bežiace na pozadí (`systemd` služby) majú voči jadru upravené užívateľské oprávnenia pre priamy prístup k sériovým portom a audio subsystému bez nutnosti dodatočných bezpečnostných „sudo“ potvrdzovaní obsulohy.

[MIESTO PRE OBRÁZOK: Fotografia zapojeného Raspberry Pi v rozvádzači s HDMI a LAN káblovou konektivitou]

4.2 Distribuované uzly – ESP32

- **Fyzický variant modulu (ESP32 DevKit V1):** Využitá je štandardná 30-pinová vývojová doska ESP32 DevKit V1, ktorá spája čip ESP-WROOM-32 s potrebnými LDO regulátormi a USB prevodníkom. Je nasadená najmä pre aplikácie presného časovania, typicky pre generovanie PWM signálov ovládajúcich **jednosmerné motory**.

- **Modul RP2350 (Waveshare Relay):** Pre uzly určené na silové spínanie svetiel a dymostrojov sa nevyužíva klasické zapojenie s relé modulmi, ale integrovaná prie-

myselná doska od firmy Waveshare, osadená priamo mikrokontrolérom architektúry **RP2350**. Tieto uzly sa špecializujú na oddelené 230 V okruhy.

- **Spôsob plošnej strediskovej montáže:** Tieto samostatné bloky sa osadzujú lokálne v múzeu priamo k zariadeniam (blízko motorov či senzorov). Moduly sa spoliehajú na samostatné napäťové zdroje, dôležité je len pripojenie na jednotnú lokálnu Wi-Fi/LAN sieť, čím odpadá potreba ťahať zväzky logických vodičov k centrálnemu Raspberry Pi.

[MIESTO PRE OBRÁZOK: Fotografia modulu ESP32 ESP-WROOM-32 a jeho GPIO rozloženia]

4.3 Výkonové prvky

Súbor elektronických komponentov zabezpečujúcich fyzický prejav asynchrónnych inštrukcií.

4.3.1 Riadenie jednosmerných (DC) motorov

Aplikácie asynchrónneho otáčania koncových fyzických prvkov (napr. otvárania mechanických zámkov a dvier inštalácie) sú realizované *DC motormi*. Zariadenie prekladajúce signálovú logiku mikrokontroléra na výkonovú prúdovú linku z motorového zdroja napätia využíva technológiu H-mostíka.

Implementačné detaily motorických obvodov:

- **Voľba výkonového budiča BTS7960:** Pre spoľahlivé ovládanie 12 V a 24 V jednosmerných motorov sme z dôvodu nevyhovujúcej tepelnej únosnosti bežne dostupných starších obvodov (napr. L298N) aplikovali robustný vysokoprúdový H-mostík **BTS7960**. Tento dvojitý modulárny menič s priamym riadením logikou dosahuje špičkovú prúdovú zatažiteľnosť **až 43 A**, čím hravo znáša drsné kinetické rázy veľkých inštalačných motorov (napr. stacionárne motory navijákov a pák v únikových miestnostiach). Zapojenie vyžaduje iba 4 komunikačné vodiče (`GND`, `5V`, `PWM1`, `PWM2`).

- **Progresívne ovládanie PWM (Impulzná šírková modulácia):** Uzol zabezpečujúci pohyb (`controller_MOTORS`) nevykonáva len binárne zapnutie a vypnutie (On/Off logiku). Systém využíva technológiu hardvérového PWM regulovania otáčok pomocou premenlivej striedy signálu. Hardvérový časovač v ESP32 beží na prednastavenej vysokej pracovnej frekvencii 20 kHz, čím sa eliminuje nežiaduce počuteľné pískanie cievok motora.

- **Prevencia proti trhaniu (*Smooth ramping*):** Pre eliminovanie prúdových nárazov na mechanické prevodovky rekvizít a samotné H-mostíky má systém vo firmvéri zakomponovaný plynulý nábeh a výbeh hodnoty PWM. Tieto prechody sú definované krokmi (`SMOOTH_STEP`) a časovými oneskoreniami (`SMOOTH_DELAY`), čo rapídne predlžuje servisné cykly muzeálnych rekvizít namáhaných dennou prevádzkou.

[MIESTO PRE OBRÁZOK: Použitý výkonový H-mostík BTS7960 s dimenzovaným chladičom pripojený k motoru a ESP32]

4.3.2 Spínanie sieťového napätia (230 V) a senzorika

Pre riadenie spotrebičov, ktoré nepracujú s plynulým jednosmerným chodom, ale spúšťajú vizuálne efekty či scénické nasvietenie s veľkým prúdovým odberom zo sieťového napätia, bola zvolená reléová spínacia stratégia. Paralelne je doriešený záchyt senzorických vstupov od účastníkov.

- **Priemyselný reléový modul:** Okamžité zapnutie bodových *halogen* reflektorov alebo 230 V hmlových dymostrojov si vyžiadalo inštaláciu skompletizovaného a bezpečného riešenia. Využitý bol **8-kanálový priemyselný relé modul (od spoločnosti Waveshare)**. Zaujímavosťou tohto uzla je, že na rozdiel od klasických modulov neriadených priamo ESP32 čipom, tento špecifický modul stavia na integrovanom dvojjadrovom mikrokontroléri **RP2350** (architektúra Raspberry Pi). Okrem 8 nezávislých relé (každý s kontaktným zaťažením do 10 A / 250 V AC) modul plne podporuje káblovú sieť Ethernet prostredníctvom zbernice W5500 a štandard PoE (*Power over Ethernet*), čím sa pre túto kritickú silovú vetvu úplne eliminuje závislosť na lokálnom Wi-Fi pokrytí.
- **Optické oddelenie obvodov:** Z dôvodu najvyššej možnej galvanickej bezpečnosti sa vybraný reléový blok do riadiacej cesty pripája výlučne cez zapuzdrené **optočleny (*optocouplers*)**. Pomocou vstavaného generátora infračerveného žiarenia v čipe prebieha úplné fyzické oddelenie malonapäťovej riadiacej logiky od životu nebezpečného potenciálu 230 V na spínacích kontaktoch, čím sa absolútne zamedzuje zničeniu mikrokontroléra pri prerazení izolácie na záťaži.
- **Senzorický prístup obsluhy s filtrom:** Na získavanie asynchrónnych vstupov („zmysly“) sa využívajú záchytné tlačidlá (napr. `Trigger Button` na pine GPIO 32). Každý signál z fyzického dotyku je firmvérom ESP32 hmatateľne prefiltrovaný. Aplikovaný je hardvérový **debounce časovač** v trvaní typicky 60 ms, ktorý eliminuje elektromagnetické mechanické zákmity kontaktov na prečítaných pinoch. Dodatočne je zavedený aj aplikačný **cooldown** filter (napríklad 4000 ms) zabraňujúci cyklenému odosielaniu stoviek MQTT správ pri opakovanom nekontrolovateľnom stláčaní tlačidla používateľom expozície.

4.4 Napájanie a zapojenie prototypu

Prevádzka komplexnej mechatronickej sústavy pozostávajúcej z desiatok distribuovaných modulov vyžaduje dodržiavanie prísnej vrstvenej izolácie napätí. Kvôli masívnym rozdielom vo výkone by akékoľvek zlúčenie signálových hladín s vysokoodberovým napájaním z jedného nekontrolovaného meniča viedlo k nestabilitám a resetom riadiacej logiky. Vzorový distribuovaný uzol musí pre spoľahlivú prevádzku využívať nasledujúce tri fundamentálne kategórie odberov napätí:

- **Hladina logiky a riadiacich signálov (3.3 V):** Hladina predstavujúca výhradne bezodberový referenčný tok signálov. Primárne napájanie čipov z radu *ESP32* vrá-

tane generovaného PWM signálu na pinoch či komunikácie na zbernici štandardu *I2C*. Na tomto potenciáli je absolútne vylúčené prevádzkovať napájanie akéhokoľvek vonkajšieho pohybu (okrem interných diód v optickom člene). Na napájanie vnútorného LDO regulátora vývojovej dosky sa pristupuje klasickým 5 V prívodom s využitím oddelených prepojovacích „ground“ zemných prekovov pre uzavretie uzlov a tolerancií s rozširujúcimi periférami.

- **Pracovná hladina snímačov a zosilňovačov (5.0 V):** Predstavuje ideálny zlaďovací napäťový uzol vhodný pre chod centrálneho mikropočítača Raspberry Pi a napájanie primárnej cievkovej dosky modulov od *Waveshare*. Do izieb sa spravidla dodáva zo značkových sieťových adaptérov s USB-C konektivitou eliminujúcou prepúšťanie napäťového šumu. Rovnako prislúcha a funguje aj pre terminály ESP32 pomocou pinov `VIN`.
- **Silová prúdová rovina pre aktuátory (12 V / 24 V / 230 V):** Na rovine fyzických inštalácií v modeloch sú aktívne silové toky koncového výkonu pre chod komutátorov (cez svorkovnicu na drivere L298N priamo do statora), výkonného audia či halogénových svetiel. Tieto prúdy pochádzajú z hrubo dimenzovaných dedikovaných a stabilizovaných priemyselných spínaných zdrojov umiestnených v rozvodnej skrini (v rozvádzacej vrstve) s dostatočným rezervným prúdovým dimenzovaním pre rozbehové javy motorov.

[MIESTO PRE OBRÁZOK: Elektrotechnická schéma zapojenia prototypu (napäťové úrovne 3.3V, 5V, 12V/230V, optické oddelenie relé a mosfety)]

5 Implementácia softvérovej časti

Táto kapitola opisuje programové vybavenie, so zreteľom na využitie systémového prístupu a minimalizáciu čistých výpisov kódu v prospech názorných blokových schém.

5.1 Firmvér ESP32

Všetky tri firmvérové varianty sú implementované v jazyku C++ využívajúc framework Arduino a zdieľajú spoločný inicializačný základ. Pri štarte sa každý uzol pripojí na nakonfigurovanú Wi-Fi sieť, autorizuje sa voči centrálnemu MQTT brokeru, zverejní zostatkovú (*retained*) správu `online` na diagnostický topik a nastaví LWT správu `offline` pre prípad výpadku spojenia. Konfigurácia (SSID, heslo, adresa brokera, názov topiku) je vyčlenená do samostatného súboru `config.cpp`, čo umožňuje nasadiť rovnaký kód na viacero fyzických kusov bez úpravy hlavného zdrojového súboru.

Jednotlivé varianty sa od seba líšia predplatenými topikmi a spôsobom mapovania prijatých príkazov na fyzické výstupy:

- **Motorický uzol (`esp32_mqtt_controller_MOTORS`):** Uzol sa prihlasuje na topiky `room1/motor1`, `room1/motor2` a `room1/STOP`. Prijatý textový príkaz (napr. `ON:120:L` alebo `SPEED:80`) je rozparsovaný a preložený na hardvérové PWM signály generované na frekvencii 20 kHz s 8-bitovým rozlíšením (0–255). Pre elimináciu prúdových rázov pri pohybe mechaniky je implementovaný plynulý nábeh a dobeh otáčok (*smooth ramping*) s konfigurovateľným krokom a oneskorením. Po vykonaní príkazu uzol odošle potvrdenie na topik s príponou `/feedback`.

- **Reléový uzol (`esp32_mqtt_controller_RELAY`):** Uzol ovláda až 8 samostatných reléových kanálov (osvetlenie, dymostoj a ďalšie spotrebiče). Zariadenia sú adresované topikmi v tvare `room1/<meno>` (napr. `room1/light/1`) alebo skupinovo cez `room1/effects/#`. Fyzické výstupy sú riadené buď priamo cez GPIO piny, alebo cez I2C expandér (adresa `0x20`, piny SDA/SCL: `42/41`), čo umožňuje rozšíriť počet kanálov bez zmeny základného kódu. Pridanie nového zariadenia si vyžaduje iba doplnenie záznamu do konfiguračného poľa `DEVICES[]`.

- **Tlačidlový uzol (`esp32_mqtt_button`):** Najjednoduchší variant slúži výhradne ako senzorický vstup. Pri stlačení tlačidla (GPIO pin 32) uzol zverejní správu `START` na topik `room1/scene`, čím spustí predvolenú scénu na centrále. Firmware obsahuje hardvérový debounce filter (60 ms) a aplikačný cooldown (4000 ms) zabraňujúci opakovanému odosielaniu triggra pri dlhšom stlačení.

[MIESTO PRE OBRÁZOK: Bloková schéma spoločného firmvérového základu a rozvetvenej špecializácie troch ESP32 variantov]

5.2 Backend Raspberry Pi

Hlavný proces je naprogramovaný v jazyku Python a využíva asynchrónne programovanie.

- **Hlavná trieda (`MuseumController`):** Koordinuje beh iných vlákien a drží inštanciu MQTT klienta.
- **Inverzia závislostí (`ServiceContainer`):**
  - Namiesto tvrdého prepájania tried využíva architektúra kontajner služieb.
  - Pri štarte dynamicky načítava handlery pre audio, video a parser scén, čím zabezpečuje maximálnu softvérovú modularitu zadania.

[MIESTO PRE OBRÁZOK: UML Diagram tried (Class diagram) pre MuseumController a dynamický ServiceContainer]

5.3 Stavový automat a scene engine

Riadiaca logika expozície nahrádza tradičné procedurálne podmieňovanie konečným automatom.

- **Reprezentácia stavov:** Každá logická časť výstavy je jedným objektom obsahujúcim zásobníky úloh (`onEnter`, `timeline`, `transitions`).
- **Asynchrónny prechod udalosťami:**
  - Engine nie je zablokovaný čakaním v slučke (‘delay‘).
  - Prechody sú vyhodnocované reaktívne vo forme tzv. *Eventov* (udalostí) prijímaných z frontend dashboardu alebo od senzorických ESP32 uzlov cez MQTT správu.
  - Ak je podmienka triggra splnená, `TransitionManager` zruší bežiace asynchrónne časovače v súčasnom stave a prepne automat do stavu nového.

5.4 Webový dashboard

Riadiace centrum pre personál múzea, prístupné z tabletu alebo mobilu vizuálnou formou.

- **Tech stack:** Využitie Python mikroframeworku **Flask** a asynchrónnej obojsmernej komunikácie cez **Socket.IO**.
- **Funkcionality operátora:**
  - **Monitoring:** Živé zobrazenie pripojených ESP32 uzlov a aktuálne prebiehajúceho stavu FSM.
  - **Manuálne zásahy:** Možnosť interaktívne spustiť/zastaviť scénu (*Master Override*) a núdzovo zablokovať prúdy do motorov.
  - Čítanie serverových chybových logov prichádzajúcich v reálnom čase zo `ServiceContainer`-a.

5.5 Konfigurácia scén – JSON formát

Umožňuje editáciu scén bez nutnosti rekompilácie jadra.

- **Formát uzlov:** Koreň s atribútom `initialState` a poľom `states`, ktoré obsahuje abstraktné definície choreografie.
- **Polia vnútri stavu:**
  - `onEnter`: Priame pole akcií (napr. mqtt príkazy alebo spustenie zvuku s `audio: "file.wav"`).
  - `timeline`: Pole objektov `delay` s `action`.
  - **Vetvenie:** Ukážka poľa `transitions`, kde uzol `goto` definuje ďalší stav na základe prijatia asynchrónnej zhody typu `message` od konkrétneho aktéra.

[MIESTO PRE VÝPIS KÓDU/SCHÉMU: Zjednodušená vizualizácia štruktúry konfiguračného JSON súboru]

6 Experimentálne overenie

Táto kapitola prezentuje reálne skúšky navrhnutého systému, ktoré overujú splnenie stanovených cieľov zadania v podmienkach reálnej *runtime* prevádzky.

6.1 Popis demonštračnej úlohy (Setup)

- **Fyzický prototyp:** Systém bol priamo nainštalovaný a oživený na konkrétnom reálnom exponáte v priestranstve expozície.

- **Zapojené uzly:**

  – Raspberry Pi s bežiacim centrálnym procesom `MuseumController` a pripojenou audiovizuálnou technikou.

  – ESP32 Motorový uzol pripojený na lokálny jednosmerný pohon.

  – ESP32 Reléový uzol spínajúci scénické osvetlenie miestnosti.

6.2 Testované scenáre a výsledky

Overenie stability systému prebiehalo v troch izolovaných rovinách: prevádzka s hardvérom, softvérová výdrž jadra a odolnosť voči sieťovým výpadkom.

6.2.1 Scenár 1 – Komplexná lineárna scéna s hardvérom

- **Cieľ:** Otestovať plynulosť a asynchrónnu presnosť prechodov definovaných v JSON súbore počas riadenia fyzických aktuátorov priamo v múzeu.

- **Priebeh:** Systém musel prehrať multimediálny audiovizuálny obsah súčasne s presne načasovaným pohybom motorov a dynamickým zhasínaním svetiel.

- **Výsledok:** Všetky akcie boli spustené v časoch definovaných v *timeline*. Komunikácia s mikrokontrolérmi prebehla bez zaznamenaného oneskorenia a synchronizácia audio-video stopy so svetelnými efektmi bola vizuálne konzistentná počas celého trvania testu.

6.2.2 Scenár 2 – Dlhodobý test stability riadiaceho systému

- **Cieľ:** Overiť odolnosť Raspberry Pi a MQTT brokera proti pretečeniu pamäte a zlyhaniu pri dlhodobom plynulom behu (tzv. *Soak test*).

- **Priebeh:** Centrálny program systému bežal na stole formou *dry-run* testovania v kuse niekoľko dní v laboratórnych podmienkach. Priebežne dochádzalo k ojedinelému spúšťaniu scén a monitorovaniu odosielaných MQTT príkazov na zbernicu.

- **Výsledok:** Proces `MuseumController` zostal počas celej doby testu bez reštartu. Príkazy boli generované na MQTT zbernicu priebežne bez výpadku, čo preukázalo dostatočnú stabilitu softvéru pre nepretržitú prevádzku.

6.2.3 Scenár 3 – Test sieťovej odolnosti a LWT (Last Will)

- **Cieľ:** Overiť detekčný a oznamovací mechanizmus systému pri neštandardnej strate bezdrôtového spojenia uzlov na Wi-Fi sieti.
- **Priebeh:** Koncovým ESP32 uzlom (*Controller_MOTORS* a *Controller_RELAY*) bolo striedavo fyzicky odpájané a následne vracané napájanie za behu celého systému.
- **Výsledok:** Po obnovení napájania sa uzly autonómne pripojili späť na Wi-Fi sieť a MQTT brokera. Dashboard správne zaznamenal zmenu stavu — prepis indikátora z `offline` na `online` prebehol na základe LWT správy od brokera a následného `online` hlásenia od uzla.

6.3 Zhodnotenie overenia

- **Záver experimentov:** Všetky tri testované scenáre boli úspešne overené. Architektúra distribuovanej hviezdicovej siete sa ukázala ako dostatočne stabilná pre podmienky plného fyzického zaťaženia v prostredí expozície.
- Bezdrôtová komunikácia cez protokol MQTT preukázala reaktivitu postačujúcu na spracovanie hardvérových triggerov bez merateľného oneskorenia, čo vylúčilo potrebu prechodu na drôtové prepojenie.

7 Záver

Záverečné zhrnutie práce posudzujúce, či a ako sa podarilo naplniť pôvodné ciele zadania.

- **Ciele a dosiahnutý stav:**
  - Hlavným cieľom práce bolo navrhnúť modulárny riadiaci systém pre múzejné expozície, ktorý umožní synchrónne ovládanie fyzických efektov (motory, 230 V spotrebiče, osvetlenie) a multimediálneho obsahu z jediného centrálneho uzla.
  - Realizácia potvrdila úspešnú fúziu výpočtovo silného Raspberry Pi a distribuovaných ESP32 sieťových uzlov do jednej funkčnej virtuálnej hviezdy.
- **Hlavné prínosy riešenia:**
  - Najväčším prínosom je odstrihnutie tvrdej programátorskej aplikačnej logiky (Python/C++) od samotného chodu výstavy. Vďaka aplikácii inžinierskeho konečného automatu (FSM) a používaniu textového kontraktu v `.json` formáte dokáže muzeálny personál upravovať časy, kroky a audiovizuálny obsah výstavy bez nutnosti vedieť programovať či využívať drahé licencie priemyselných PLC.
- **Výsledky overenia technológie:**
  - Reálne záťažové testy na exponáte deklarovali, že Publish/Subscribe model (MQTT nad lokálnou LAN/Wi-Fi sieťou) netrpí pre výstavu vnímateľnými reaktančnými oneskoreniami.
  - Architektúra bez problémov prechádzala dlhodobými *Soak* testami a vďaka LWT packetom úspešne ošetrila fyzické odpájanie uzlov.
- **Nedostatky a obmedzenia:**
  - Bezpečnostná vrstva Wi-Fi siete v reálnom múzeu musí byť prevádzkovo dedikovaná len pre exponát, inak hrozí rušenie spôsobené návštevníkmi (kolízie frekvenčného pásma).
  - Nutnosť ručného hardcodovania `BASE_TOPIC_PREFIX` v C++ firmvéri pre každý nový fyzický kus ESP32 mikročipu pri úvodnom *flashovaní*.
- **Návrhy na budúci rozvoj:**
  - Prechod do plne lokalizovanej webovej administrácie (React Dashboard), odkiaľ bude možné tvoriť JSON scény vizuálnym presúvaním blokov (tzv. „Node-Based Editor“).
  - Možnosť plnohodnotnej over-the-air (OTA) aktualizácie firmvéru priamo prostredníctvom odoslania binárneho príkazu cez `Mosquitto` hub na konkrétny MQTT `device id` uzol.

Zoznam obrázkov

2.1 Centralizovaná topológia 11
2.2 Distribuovaná topológia 11

Zoznam tabuliek

2.1 Úrovne kvality služby (QoS) v MQTT 14

Použitá literatúra

[1] AL-JUMAILY, M. Automation in industry using Programmable Logic Controllers (PLCs). *International Journal of Computer and Communication Engineering*. 2015, roč. 6, s. 924. Dostupné z: https://www.computersciencejournals.com/ijccn/article/83/6-1-1-924.pdf

[2] BLIIOT. Why Node-Red is the Best IoT Development Tool? *Industrial IoT Gateway Blog* [online]. 2024 [cit. 2025-02-20]. Dostupné z: https://bliiot.com/info-detail/why-node-red-is-the-best-iot-development-tool

[3] BPAV TECHNOLOGY. Crestron vs Extron vs AMX: What Suits Your Project? *BPAV Blog* [online]. 2023 [cit. 2025-02-20]. Dostupné z: https://blog.bpav.global/news-insights/crestron-vs-extron-vs-amx-what-suits-your-project

[4] SIMPSON, John Huntington. *Show Networks and Control Systems*. 2. vyd. New York: Entertainment Technology Press, 2017. ISBN 978-1138090874.

[5] MQTT.ORG. MQTT: The Standard for IoT Messaging [online]. 2024 [cit. 2025-02-20]. Dostupné z: https://mqtt.org/

[6] CAVLI WIRELESS. MQTT Protocol in IoT: A Guide to Reliable IoT Communication. *IoT Blog* [online]. 2023 [cit. 2025-02-20]. Dostupné z: https://www.cavliwireless.com/blog/nerdiest-of-things/what-is-the-mqtt-protocol

[7] CELERDATA. YAML, JSON and XML: A Practical Guide to Choosing the Right Format. *CelerData Blog* [online]. 2024 [cit. 2025-02-20]. Dostupné z: https://celerdata.com/glossary/yaml-json-and-xml-a-practical-guide-to-choosing-the-right-format

[8] AL-SHUHAIB, T. Low-Cost ESP32, Raspberry Pi, Node-Red, and MQTT Protocol Based SCADA System. *IEEE International Conference on Intelligent Systems*. 2020. Dostupné z: https://www.researchgate.net/publication/347154219

[9] SOMERVILLE, I. *DMX512 Stage Lighting Basics*. Scribd Document [online]. 2013 [cit. 2025-02-20]. Dostupné z: https://www.scribd.com/document/132762444/DMX-Beginners-Guide-Rev2

[10] MILLS, D. *Network Time Protocol (Version 4)*. RFC 5905. Internet Engineering Task Force, 2010. Dostupné z: https://datatracker.ietf.org/doc/html/rfc5905

[11] CLOUDWAYS. Django vs Flask: The Best Python Web Framework in 2025? *Cloudways Blog* [online]. 2024 [cit. 2025-02-20]. Dostupné z: https://www.cloudways.com/blog/django-or-flask/

[12] INFINEON TECHNOLOGIES. *BTS7960 High Current PN Half Bridge Datasheet* [online]. 2004. Dostupné z: https://www.infineon.com/dgdl/Infineon-BTS7960-DS-v01_01-en.pdf